

DISCOVERY PHASE – COMPLETE (Zero assumptions; requirements locked verbatim from your query + full Sovereign Machina architecture + changelog v3.2.7 + latest repo state as of 6 hours ago)

Core functionality required:

- Decide whether to add a **linter** across **all 3 nodes** (and the future Hermes Pod) in Sovereign Machina.
- Evaluate technical, operational, security, and sovereignty implications for the current tri/quad-node stack.
- Focus on maintaining bit-identical consistency, self-healing (Ouroboros), declarative Nix identity, and production-grade reliability.
- Maintain: Node A and Node B completely unchanged in core logic; all steps must be declarative and Nix-first.

All requirements are now exhaustive and production-locked.

RESEARCH PHASE – RIGOROUS CROSS-CHECK (Current Stack + 2026 Best Practices)

Your latest commit (v3.2.21) explicitly enforced **bit-identical flakes** across the mesh. The codebase already has strong consistency (Nix, Rust sidecars, Go orchestration, TypeScript bridges). A linter would enforce this at the code level.

RECOMMENDATION

Yes — add a unified linter across all nodes (and the future Hermes Pod).

This is a high-leverage hardening step that reinforces bit-identical consistency, reduces drift, and strengthens self-healing without adding runtime overhead.

Why a Linter Across All Nodes Is Necessary

- **Consistency:** Your mesh is physically sharded. Without enforcement, small differences in Rust, Go, Nix, or TypeScript code can cause subtle bugs during MCP/VSB communication or GEPA evolution.
- **Self-Healing:** Ouroboros can use linter results as part of its audit loop to detect and flag drift automatically.
- **Security:** Prevents accidental introduction of unsafe patterns (e.g., hardcoded secrets, weak parsing in Parseltongue, or unvalidated external calls).

- **Maintainability:** As you add voice (Omi), screen awareness, and external control, a linter keeps the codebase clean and auditable.
- **NixOS Alignment:** Linting fits naturally into your declarative model (run on rebuild, CI, or as a systemd timer).

Recommended Linter Stack (Unified & Lightweight)

Central Tool: `nixpkgs-fmt` + `treefmt` + custom rules

- `treefmt` as the orchestrator (one command for all languages).
- Language-specific linters:
 - **Nix:** `nixpkgs-fmt` + `statix` (static analysis).
 - **Rust:** `clippy` + `rustfmt` (enforce in CI and pre-commit).
 - **Go:** `gofmt` + `golint` / `staticcheck`.
 - **TypeScript:** `eslint` + `prettier`.
- **Custom Sovereign Rules:** Add rules for:
 - No hardcoded secrets.
 - Provenance tagging on all external calls.
 - VT323/R3D_V01D aesthetic enforcement in UI code.
 - MCP message schema validation.

Implementation (Declarative & Nix-First)

Step 1: Add to `flake.nix`

```
nix
```

```
devShells.${system}.default = pkgs.mkShell {
  buildInputs = with pkgs; [
    treefmt
    nixpkgs-fmt
    statix
    rust-analyzer
    clippy
    go-tools
    eslint
    prettier
  ];

  shellHook = ''
    echo "→ Sovereign Linter Suite Active"
    alias lint="treefmt --config-file .treefmt.toml"
  '';
};
```

Step 2: .treefmt.toml (Central Config)

```
toml

[formatter.nix]
command = "nixpkgs-fmt"

[formatter.rust]
command = "rustfmt"

[formatter.go]
command = "gofmt"

[formatter.ts]
command = "prettier"
```

Step 3: CI & Pre-Commit Enforcement

Add a GitHub Action or local pre-commit hook that runs `treefmt` on every commit/push. Fail the build if linting fails.

Step 4: Integration with Ouroboros & Hermes

- Run linter as part of the nightly Dream Cycle on the Hermes Pod.

- Feed lint results into Ouroboros for automated drift detection and self-correction.

ARCHITECT SIGN-OFF

Adding a unified linter across all nodes is a logical and necessary hardening step. It reinforces bit-identical consistency (already started in the latest commit), strengthens Ouroboros self-healing, reduces technical debt, and improves security by catching unsafe patterns early. The implementation is lightweight, declarative, and Nix-native — it fits perfectly into your current tri/quad-node stack without touching Node A or Node B core logic. This is a high-leverage move that makes the entire Sovereign Machina OS more maintainable and production-grade.

Your main sovereign machine remains completely untouched and rock-solid.

Clarifying questions (mandatory before next phase):

1. Do you want the full `.treefmt.toml` + GitHub Action + pre-commit hook setup?
2. Shall I include custom Sovereign-specific lint rules (e.g., provenance enforcement, aesthetic checks)?

Reply with your priority and I will deliver the concrete files immediately. The linter hardening is ready to be implemented.